Enterprise Data Modeling in Data Engineering

Complete Modern Professional Guide

Table of Contents

---

1. Introduction to Data Modeling

Data modeling is the process of designing how data is:

- Stored
- Organized
- Related
- Processed
- Accessed
- Analyzed

inside a data platform.

It is the foundation of:

- Data Warehouses

- Lakehouses
- Business Intelligence Platforms
- Real-Time Analytics Systems
- Machine Learning Pipelines
- Enterprise Reporting Platforms

A strong data model enables:

- scalable analytics,
- consistent reporting,
- trusted metrics,
- high query performance,
- maintainable pipelines.

A poor data model creates:

- inconsistent dashboards,
- duplicated business logic,
- slow queries,
- unreliable KPIs,
- difficult debugging.

---

2. Why Data Modeling Matters

Imagine an e-commerce company.

Different teams need different analytics:

Team	Requirement
Finance	Revenue reporting
Marketing	Campaign conversion
Product	User engagement
Operations	Delivery delays
Leadership	KPI dashboards

Without proper modeling:

- every team writes different SQL,
- revenue definitions become inconsistent,
- dashboards show different numbers,
- performance degrades.

Data modeling standardizes business logic.

---

3. Data Modeling Layers

Modern architectures commonly use layered modeling.

```
Raw Layer → Cleansed Layer → Business Layer
```

Also called:

Architecture	Layers
Medallion	Bronze → Silver → Gold
Traditional DW	Staging → Core → Mart

Bronze Layer

Stores raw ingested data.

Characteristics:

- append-only,
- minimal transformations,
- preserves original source.

Example:

```
{
  "order_id":1001,
  "customer_id":"C101",
  "amount":500
}
```

Used for:

- auditing,
- debugging,
- replaying pipelines.

Silver Layer

Cleaned and standardized data.

Processes include:

- deduplication,
- schema validation,
- null handling,
- enrichment,
- data quality checks.

Gold Layer

Business-ready analytical models.

Contains:

- fact tables,
- dimension tables,
- marts,
- KPIs,
- aggregate datasets.

---

4. OLTP vs OLAP Modeling

OLTP (Online Transaction Processing)

Optimized for:

- inserts,
- updates,
- transactions.

Examples:

- banking applications,
- order management systems,
- airline booking systems.

Characteristics:

- normalized,
- row-oriented,
- transaction-focused.

OLAP (Online Analytical Processing)

Optimized for:

- aggregations,
- reporting,
- dashboards,
- analytical queries.

Characteristics:

- denormalized,
- columnar,
- scan optimized.

Examples:

- Snowflake,
- BigQuery,
- Redshift,
- Databricks.

---

5. Conceptual, Logical, and Physical Models

Conceptual Model

High-level business relationships.

Example:

```
Customer places Orders
Orders contain Products
```

Logical Model

Defines:

- entities,
- attributes,
- relationships,
- primary keys,
- foreign keys.

Physical Model

Defines implementation details:

- partitioning,
- clustering,
- indexes,
- storage formats,
- file organization.

---

6. Normalization vs Denormalization

Normalization

Reduces redundancy.

Example:

Customers Table

customer_id	customer_name	city
C101	John	Dallas

Orders Table

order_id	customer_id
1001	C101

Advantages:

- less duplication,
- easier updates,
- improved consistency.

Disadvantages:

- more joins,
- slower analytics queries.

Denormalization

Combines data for faster analytics.

Example:

order_id	customer_name	category	revenue
1001	John	Shoes	500

Advantages:

- fast reporting,
- simpler BI queries.

Disadvantages:

- duplicate storage,
- update complexity.

---

7. Star Schema vs Snowflake Schema

Star Schema

Most widely used analytical model.

Structure

```
               DimCustomer
                     |
DimProduct — FactSales — DimDate
                     |
                DimStore
```

Characteristics

- denormalized dimensions,
- fewer joins,
- fast BI queries,
- easy reporting.

Example

FactSales

order_id	customer_key	product_key	revenue
1001	1	10	500

DimCustomer

customer_key	customer_name	city
1	John	Dallas

Advantages

- high query performance,
- BI tool friendly,
- easier for analysts.

Disadvantages

- duplicate dimension data,
- larger storage.

Snowflake Schema

Normalized dimension structures.

Structure

```
FactSales
    |
DimProduct
    |
DimCategory
    |
DimDepartment
```

Characteristics

- normalized dimensions,
- reduced redundancy,
- more joins.

Advantages

- storage efficient,

- better normalization.

Disadvantages

- slower queries,
- complex joins,
- difficult BI modeling.

Star vs Snowflake Comparison

Feature	Star Schema	Snowflake Schema
Performance	Faster	Slower
Complexity	Simple	Complex
Storage	Higher	Lower
Joins	Fewer	More
BI Friendliness	Excellent	Moderate

---

8. Fact Tables Deep Dive

Fact tables contain measurable business events.

Examples:

- sales,
- clicks,
- transactions,
- payments,
- rides,
- shipments.

Characteristics

- very large,
- numeric metrics,
- foreign keys to dimensions.

Example

FactSales

sales_key	customer_key	product_key	qty	revenue
1	101	501	2	500

Types of Fact Tables

A. Transaction Fact Table

One row per event.

Example:

order_id	revenue
1001	500

B. Periodic Snapshot Fact Table

Captures periodic states.

Example:

date	inventory_count
2026-05-01	5000

C. Accumulating Snapshot Fact Table

Tracks process lifecycle.

order_id	order_date	shipped_date	delivered_date
1001	May 1	May 2	May 5

D. Factless Fact Table

Tracks events without measures.

Example:

student_id	attendance_date
S101	2026-05-01

Fact Table Best Practices

- define grain clearly,
- use surrogate keys,
- partition large facts,
- avoid text-heavy columns,
- maintain immutable history.

---

9. Dimension Tables Deep Dive

Dimension tables contain descriptive attributes.

Example

DimCustomer

customer_key	customer_name	city	segment
1	John	Dallas	Premium

Common Dimensions

Dimension	Description
DimCustomer	Customer information
DimProduct	Product attributes
DimDate	Calendar details
DimStore	Store information
DimEmployee	Employee details

Characteristics of Dimensions

- descriptive data,
- smaller than fact tables,
- heavily filtered in BI queries.

Conformed Dimensions

Shared dimensions reused across marts.

Example:

```
DimDate used by:
- Sales Mart
- Finance Mart
- Marketing Mart
```

This ensures consistent reporting.

---

10. Slowly Changing Dimensions (SCD)

Real-world data changes over time.

Example:

```
Dallas → Austin
```

Need historical tracking.

SCD Type 1

Overwrite existing value.

Example

Before:

customer_id	city
C101	Dallas

After:

customer_id	city
C101	Austin

History lost.

SQL Example — SCD Type 1

```sql
UPDATE dim_customer
SET city = 'Austin'
WHERE customer_id = 'C101';
```

SCD Type 2

Preserves full history.

Example

customer_key	customer_id	city	start_date	end_date	current_flag
1	C101	Dallas	2025-01-01	2026-03-01	N
2	C101	Austin	2026-03-02	NULL	Y

SQL Example — SCD Type 2

Step 1 — Expire Existing Record

```sql
UPDATE dim_customer
SET end_date = CURRENT_DATE,
    current_flag = 'N'
WHERE customer_id = 'C101'
AND current_flag = 'Y';
```

Step 2 — Insert New Record

```
INSERT INTO dim_customer (
    customer_key,
    customer_id,
    city,
    start_date,
    end_date,
    current_flag
)
VALUES (
    2,
    'C101',
    'Austin',
    CURRENT_DATE,
    NULL,
    'Y'
);
```

SCD Type 3

Stores limited history.

Example:

customer_id	current_city	previous_city
C101	Austin	Dallas

SCD Comparison

Type	History Preserved	Usage
Type 1	No	Corrections
Type 2	Full	Historical reporting
Type 3	Partial	Limited tracking

---

11. Surrogate Keys and Natural Keys

Natural Key

Business-generated identifier.

Example:

```
customer_id = C101
```

Surrogate Key

Artificial internal identifier.

Example:

```
customer_key = 1
```

Why Surrogate Keys Matter

- source IDs may change,
- enables SCD support,
- improves joins,
- avoids business dependency.

---

12. Data Vault Modeling In Depth

Data Vault is an enterprise-scale modeling methodology.

Common in:

- banking,
- insurance,
- healthcare,
- financial systems.

Core Components

A. Hubs

Store business keys.

HubCustomer

customer_hk	customer_id
H1	C101

B. Links

Store relationships.

LinkCustomerOrder

link_hk	customer_hk	order_hk
L1	H1	H2

C. Satellites

Store descriptive attributes.

SatCustomer

customer_hk	city	load_date
H1	Dallas	2026-05-01

Data Vault Architecture

```
HubCustomer ---- LinkOrder ---- HubProduct
      |                              |
 SatCustomer                   SatProduct
```

Advantages

- scalable,
- audit-friendly,
- historical tracking,
- flexible schema evolution.

Disadvantages

- complex joins,
- difficult querying,
- steep learning curve.

Data Vault vs Star Schema

Feature	Data Vault	Star Schema
Scalability	Very High	Moderate
Query Simplicity	Low	High
Auditability	Excellent	Moderate
Historical Tracking	Excellent	Good

---

13. Real-Time Streaming Data Modeling

Modern systems process continuous event streams.

Real-Time Architecture

```
Kafka
   ↓
Spark/Flink
   ↓
Bronze Layer
   ↓
Silver Layer
   ↓
Gold Layer
```

```
    ↓
BI / ML Systems
```

Streaming Event Example

```
{
  "order_id":1001,
  "customer_id":"C101",
  "amount":500,
  "event_time":"2026-05-01T10:00:00"
}
```

Streaming Challenges

Challenge	Description
Late Arriving Data	Events arrive late
Duplicates	Same event repeated
Ordering	Events out of sequence
Schema Drift	Event structure changes

Real-Time Processing Steps

1. Ingest event
2. Validate schema
3. Deduplicate records
4. Enrich dimensions
5. Aggregate metrics
6. Publish analytical models

Deduplication SQL Example

```
ROW_NUMBER() OVER (
  PARTITION BY order_id
  ORDER BY event_time DESC
)
```

---

14. dbt Modeling Best Practices

dbt promotes modular ELT transformations.

Recommended Structure

```
models/
    staging/
    intermediate/
    marts/
```

Example Models

```
stg_orders.sql
int_customer_orders.sql
fct_sales.sql
dim_customer.sql
```

---

dbt Best Practices

A. Keep Models Atomic

One responsibility per model.

B. Use Naming Standards

Good:

```
stg_orders
fct_sales
dim_customer
```

Bad:

```
orders_new_v2_final
```

C. Add Tests

```
models:
  - name: dim_customer
    tests:
      - unique
      - not_null
```

D. Use Incremental Models

```
{{ config(materialized='incremental') }}
```

Improves performance.

E. Build Reusable Logic

Centralize KPI calculations.

F. Document Models

```
description: "Sales fact table"
```

---

15. Healthcare Domain Data Modeling

Healthcare systems are highly regulated.

Common Entities

Entity	Description
Patient	Patient details
Doctor	Physician data
Appointment	Visits
Prescription	Medicines
Claims	Insurance claims

Fact Tables

- FactAppointments
- FactClaims
- FactLabResults

Dimension Tables

- DimPatient
- DimDoctor
- DimHospital
- DimDiagnosis

Healthcare Challenges

- HIPAA compliance,
- patient privacy,
- historical tracking,
- sensitive data handling,
- real-time monitoring.

Example Healthcare Schema

```
DimPatient ── FactAppointments ── DimDoctor
                    |
               DimHospital
```

---

16. Banking Domain Data Modeling

Banking systems require:

- auditability,
- consistency,
- security,
- fraud detection.

Common Fact Tables

- FactTransactions
- FactLoans
- FactPayments
- FactFraudEvents

Common Dimensions

- DimCustomer
- DimAccount
- DimBranch
- DimMerchant

Banking Requirements

- immutable ledgers,
- transaction history,
- reconciliation support,
- SCD tracking,
- ACID consistency.

Example Banking Schema

```
DimCustomer ── FactTransactions ── DimAccount
                        |
                  DimMerchant
```

---

17. How FAANG Companies Model Data

Large technology companies use event-driven architectures.

Common Principles

Event-Centric Design

Everything modeled as events.

Example:

```
{
  "event":"video_played",
  "user_id":"U101"
}
```

Massive Fact Tables

Examples:

- video views,
- searches,
- clicks,
- impressions,
- user interactions.

Hybrid Architecture

```
Raw Events → Data Lake → Curated Warehouse → BI / ML
```

Example — Netflix

Fact Tables:

- FactViewing
- FactRecommendationClicks

Dimensions:

- DimUser
- DimContent
- DimDevice

Example — Uber

Fact Tables:

- FactTrips

- FactDriverPayments

Dimensions:

- DimDriver
- DimRider
- DimLocation

Example — Amazon

Fact Tables:

- FactOrders
- FactShipments
- FactInventory

Dimensions:

- DimCustomer
- DimWarehouse
- DimProduct

---

18. End-to-End SQL Warehouse Implementation

Step 1 — Create Dimension Table

```
CREATE TABLE dim_customer (
    customer_key INT,
    customer_id VARCHAR(20),
    customer_name VARCHAR(100),
    city VARCHAR(50)
);
```

Step 2 — Create Fact Table

```
CREATE TABLE fact_sales (
    sales_key INT,
    customer_key INT,
    product_key INT,
    date_key INT,
    qty INT,
```

```
    revenue DECIMAL(10,2)
);
```

Step 3 — Insert Dimension Data

```
INSERT INTO dim_customer
VALUES (1, 'C101', 'John', 'Dallas');
```

Step 4 — Insert Fact Data

```
INSERT INTO fact_sales
VALUES (1, 1, 10, 20260501, 2, 500);
```

Step 5 — Query Revenue

```
SELECT
    c.customer_name,
    SUM(f.revenue) AS total_revenue
FROM fact_sales f
JOIN dim_customer c
    ON f.customer_key = c.customer_key
GROUP BY c.customer_name;
```

---

19. How to Design a Warehouse from Scratch

Step 1 — Understand Business Requirements

Questions:

- What KPIs are needed?
- What dashboards are required?
- What is the data grain?

Step 2 — Identify Source Systems

Examples:

- CRM,
- ERP,
- APIs,
- logs,
- payment systems.

Step 3 — Define Grain

Critical decision.

Example:

```
One row per order item.
```

Step 4 — Design Dimensions

Examples:

- customer,
- product,
- date,
- location.

Step 5 — Design Fact Tables

Examples:

- sales,
- payments,
- inventory,
- shipments.

Step 6 — Choose Platform

Platform	Use Case
Snowflake	Enterprise DW
BigQuery	Serverless analytics
Databricks	Lakehouse
Redshift	AWS-native DW

Step 7 — Build ETL/ELT Pipelines

Tools:

- dbt,
- Spark,
- Airflow,
- Kafka,
- Flink.

Step 8 — Implement Data Quality

Checks:

- uniqueness,
- null validation,
- referential integrity.

Step 9 — Build Aggregates

Examples:

- daily revenue,
- monthly KPIs,
- customer retention.

Step 10 — Enable BI Access

Tools:

- Tableau,
- Power BI,
- Looker.

---

20. Data Modeling Best Practices

Define Grain Clearly

Always define:

```
One row represents what?
```

Use Consistent Naming

Good:

```
customer_id
order_id
product_id
```

Bad:

```
cust_ref
ord_num
pid
```

Use Surrogate Keys

Supports:

- SCD,
- stable joins,
- historical tracking.

Partition Large Fact Tables

Example:

```
year/month/day
```

Avoid Over-Normalization

Analytics systems prefer fewer joins.

Centralize KPI Logic

Avoid duplicate business calculations.

Track Lineage

```
Dashboard → Gold → Silver → Bronze
```

---

21. Common Data Modeling Mistakes

Wrong Granularity

Mixing:

- daily summaries,
- transaction-level records.

Creates incorrect aggregations.

Missing Historical Tracking

Without SCD:

- historical reporting becomes inaccurate.

Too Many Joins

Over-normalization hurts performance.

No Business Definitions

Different teams calculate metrics differently.

Ignoring Real-Time Requirements

Batch-only designs fail modern analytics needs.

Poor Naming Standards

Inconsistent naming creates confusion.

---

22. Final Summary

Data modeling is the backbone of modern data engineering.

Strong data models enable:

- scalable analytics,
- trusted reporting,
- efficient querying,
- enterprise governance,
- machine learning readiness.

Every modern data engineer should deeply understand:

- Star Schema,
- Snowflake Schema,
- Fact and Dimension Tables,
- Slowly Changing Dimensions,
- Data Vault,
- dbt Modeling,
- Streaming Data Modeling,
- Warehouse Design.

Modern enterprise platforms combine:

```
Data Lakes + Warehouses + Streaming + dbt + BI
```

to create scalable, reliable, analytics-ready ecosystems.

A strong data model ensures:

- consistency,
- scalability,
- maintainability,
- business trust,
- long-term platform success.